

Стандарты разработки визуальных форм и диалоговых окон (ДО)

> **Назначение:** единый стандарт дизайна и разработки ДО для переноса в другие проекты. Определяет внешний вид, поведение, логирование и тестирование.

>

> **Технология:** любая (C# WPF, Python, PowerShell), но внешний вид — строго по стандарту.

1. Общие принципы

1. **Окно непрозрачное**, полупрозрачность — только Title bar и элементы управления (альфа-канал `#cc` = 80% или `#80` = 50%).

2. **Title bar темнее окна** по цвету и полупрозрачный.

3. Все элементы (кнопки, прогресс-бары) — полупрозрачные с градиентами и бликами (3D-эффект).

4. **Технический журнал** обязателен для каждого ДО.

5. **Автотестирование** через `--autotest <file>` — обязательное требование.

2. Окно (Window)

Свойство	Значение
AllowsTransparency	True
WindowStyle	None (без системной рамки)
Background	#E8ECF0 (серо-голубой, непрозрачный)
ResizeMode	CanResizeWithGrip
Topmost	True
Размер по умолчанию	520×580
Скругление углов	Win32 CreateRoundRectRgn , радиус 18px (эллипс 36×36)
Тень	DropShadowEffect Color=#404040 Direction=270 ShadowDepth=4 BlurRadius=10 Opacity=0.5
Внешняя рамка	BorderBrush=#1A3A60 BorderThickness=1

Скругление углов (Win32 API)

```
IntPtr hRgn = CreateRoundRectRgn(0, 0, w + 1, h + 1, 36, 36);  
  
SetWindowRgn(hWnd, hRgn, true);  
  
DeleteObject(hRgn);
```

Пересчитывать при `SizeChanged` .

3. Title bar (заголовок окна)

Свойство	Значение
Высота	40px
Background	#807080B0 (50% deep blue-gray, полупрозрачный)
CornerRadius	17,17,0,0
Нижняя рамка	BorderBrush=#1A3A60 BorderThickness=0,0,0,1
Перетаскивание	MouseLeftButtonDown → DragMove() ; DoubleClick → ToggleMaximize()

Текст заголовка (3D-эффект)

Два слоя:

- 1. **Нижний слой:** белый (White) + смещение (Margin="3,2,0,0"), Opacity="0.9" — тень
- 2. **Верхний слой:** #1A3A60 (тёмно-синий) — основной текст

Шрифт: Segoe UI, 14px, Bold.

Кнопки управления окном

Кнопка	Символ	Действие
Свернуть	–	WindowState.Minimized
Развернуть	□	ToggleMaximize()

Заккрыть	×	Close()
----------	---	---------

Стиль: `Background="Transparent" Foreground="#1A3A60" BorderThickness="0" Cursor="Hand"` , размер 40×36px.

4. Кнопки (Glossy 3D)

Primary (ОК, Сохранить и т.п.)

`CornerRadius = 10`

Внешняя рамка: #CC0F3050 (80%)

Градиент (3-stop): #CC9DC8F0 → #CC3B7BBF → #CC1A4A7A

Блик сверху: белый градиент #E0FFFFFF → #00FFFFFF, высота 7px

Текст: White, Bold, 11px

Размер: 120×38px

Secondary (Отмена, Назад и т.п.)

`CornerRadius = 10`

Внешняя рамка: #CC404040 (80%)

Градиент: #CCD0D0D0 → #CC909090 → #CC606060

Блик сверху: белый градиент #E0FFFFFF → #00FFFFFF, высота 7px

Текст: White, Bold, 11px

Размер: 120×38px

Small (▼/▲ поиск)

Те же стили, размер 30×26px, FontSize=11.

5. Прогресс-бар (ПБ)

Свойство	Значение
Height	18px
CornerRadius (рамка)	4
CornerRadius (трек)	3
Рамка	#CC0F3050 (80%), Padding=1.5
Трек (фон)	#E2E8F0

Индикатор (заполненная часть)

- Градиент (4-stop): #CCB8D8F8 → #CC3B7BBF → #CC1A4A7A → #CC0F3050
- Правая рамка: BorderBrush=#CC1A3A60 BorderThickness=0,0,1,0
- Блик сверху: белый градиент #C0FFFFFF → #00FFFFFF
- Тень снизу: #30000000 → #80000000, высота 3px

КРИТИЧЕСКОЕ: горизонтальный объём

Блик и тень идут **по горизонтали** (слева направо), создавая эффект трубы/стержня. Вертикальный объём (блик сверху) — **запрещён**.

Текст поверх ПБ

Обязателен: подписи фаз и шагов отображаются ПОВЕРХ самого ПБ (белый текст с тенью). Обновление через `DispatcherTimer`; текст обновляется всегда, включая `Value=0`.

6. DataGrid / Таблица

Свойство	Значение
VerticalScrollBarVisibility	Visible
HorizontalScrollBarVisibility	Visible
CanContentScroll	True

AlternatingRowBackground	#F5F7FA
RowBackground	White
BorderBrush	#CBD5E0
SelectionMode	Single , FullRow

Layout — вертикальный скролл + фиксация низа

```
<Grid.RowDefinitions>

    <RowDefinition Height="Auto"/>    <!-- 0: верхняя панель -->

    <RowDefinition Height="*/>        <!-- 1: DataGrid (заполняет остаток) -->

    <RowDefinition Height="Auto"/>    <!-- 2: ProgressBar (всегда внизу) -->

    <RowDefinition Height="Auto"/>    <!-- 3: Кнопки (всегда внизу) -->

</Grid.RowDefinitions>
```

DataGrid в `Height="*/`, ПБ и кнопки в `Height="Auto"` после него. Это гарантирует:

- DataGrid скроллится вертикально при переполнении
- ПБ и кнопки всегда видны при любом размере окна

7. Поля ввода (Text/Password/Combo)

Рамка: `BorderBrush="#CBD5E0", BorderThickness=1, CornerRadius=10`

Внутри: `Padding=6,4`

Шрифт: 11px

Подпись: 10px, #4A5568

Пароль **никогда не выводится** в журнал/отчёты (в техжурнале `pwd=***`).

8. Панель поиска

Счётчик: "N - M" (#4A5568, 11px)

Поле ввода: `Height=26, FontSize=11`

Кнопки: ▼ (`search_down`) / ▲ (`search_up`) – Small Glossy 3D

Enter → `search_down`

Логика:

- При вводе текста — фильтрация строк `DataGrid`
- При нажатии ▼/▲ — навигация по совпадениям с подсветкой и скроллом
- Поиск case-insensitive по всем колонкам

9. Resize (изменение размера)

Через `WM_NCHITTEST` в `WndProc` :

- `RESIZE_MARGIN` = 8px
- Все 8 направлений: `HTLEFT`, `HTRIGHT`, `HTTOP`, `HTBOTTOM` + 4 угла
- Title bar drag: `HTCAPTION` при `Y < 42` (исключая края)

10. Цветовая схема

Элемент	Цвет
Окно (фон)	<code>#E8ECF0</code>
Title bar	<code>#807080B0</code> (50% deep blue-gray)
Внешняя рамка	<code>#1A3A60</code>
Текст заголовка	<code>#1A3A60</code> + белая тень
Рамки полей/таблицы	<code>#CBD5E0</code>
Текст	<code>#4A5568</code>
Кнопка Primary	<code>#CC0F3050</code> + градиент
Кнопка Secondary	<code>#CC404040</code> + градиент
ПБ трек	<code>#E2E8F0</code>

ПБ индикатор	#CC0F3050 + градиент
--------------	----------------------

11. Технический журнал (tech_journal)

ОБЯЗАТЕЛЬНО для каждого ДО:

- При запуске создаёт `tech_journal_yyyyMMdd_HHmmss.log` рядом с exe/скриптом
- Формат строки: `[HH:mm:ss.fff] [LEVEL] сообщение`
- Уровни: `INFO`, `ERROR`, `CLICK`, `AUTOTEST`
- Логирование потокобезопасно (lock)

Обязательные записи при запуске

```
[INFO] Window constructed
```

```
[INFO] EXE path: <путь>
```

```
[INFO] Args: <аргументы>
```

```
[INFO] Loaded <N> rows (если есть данные)
```

Обязательные записи при работе

- Взаимодействие: `[CLICK] <ok|cancel|search_down|search_up|min|max|close>`
- Ошибки: `[ERROR] <описание>` с try-catch во всех обработчиках
- Изменение размера: `[INFO] SizeChanged: <W>x<H>`

12. Автотестирование — обязательное требование

12.1. Каждое ДО должно поддерживать `--autotest`

При запуске с аргументом `--autotest <путь_к_файлу>` ДО:

1. Загружает команды из файла (построчно)
2. Выполняет их последовательно с паузой 300ms между командами
3. Все действия логируются в tech_journal
4. После выполнения всех команд — автозаккрытие окна (через 500ms)

12.2. Команды автотеста

Команда	Описание
<code>log <text></code>	Запись в tech_journal
<code>wait <ms></code>	Пауза (Thread.Sleep)
<code>click <name></code>	Клик по кнопке
<code>set_text <val></code>	Заполнить текстовое поле

set_pwd <val>	Заполнить пароль
set_search <val>	Заполнить поиск
search_down	Поиск вниз
search_up	Поиск вверх
move <x,y>	Переместить окно
resize <w,h>	Изменить размер
scroll <row,col>	Скролл таблицы
close	Закрыть окно

12.3. Три обязательных этапа тестирования

КРИТИЧЕСКОЕ ТРЕБОВАНИЕ: запрещено докладывать об успешном выполнении без прохождения всех этапов.

Этап 1 — Тест открытия

- Запустить ДО с `--autotest`
- Убедиться, что tech_journal создан
- Проверить, что нет ERROR в логе

Этап 2 — Тест новой функциональности

- Для каждого пункта из промпта — команда в автотесте
- Проверить, что функция работает без ошибок

Этап 3 — Регрессия

- Базовые функции (открытие, кнопки, закрытие) не сломаны
- Нет исключений

12.4. Формат отчёта о тестировании

Результаты тестирования:

1. ☒ Д0 открывается, tech_journal создан
2. ☒ [Функция 1] работает
3. ☒ [Функция 2] работает
4. ☒ Старый функционал не затронут

12.5. Шаблон реализации автотеста

```
// Парсинг аргументов в конструкторе

private void ParseAutoTestArgs()

{

    string[] args = Environment.GetCommandLineArgs();

    for (int i = 1; i < args.Length; i++)

        if (args[i] == "--autotest" && i + 1 < args.Length)

        {

            string file = args[i + 1];
```

```

        if (File.Exists(file))
        {
            _autoTestCommands = File.ReadAllLines(file).ToList();

            _exitAfterAutoTest = true;
        }
    }
}

// Запуск в Window_Loaded через DispatcherTimer (500ms)

if (_autoTestCommands.Count > 0)
{
    DispatcherTimer t = new DispatcherTimer { Interval = TimeSpan.FromMilliseconds(500) };

    t.Tick += (s, ev) => { t.Stop(); RunAutoTestCommand(); };

    t.Start();
}

```

13. Адаптация под проект

При переносе в другой проект:

1. Сохраните цветовую схему и размеры — это эталон СТАНДАРТ2.
2. Адаптируйте команды автотеста под свои элементы управления.
3. Настройте формат tech_journal под свою систему логирования.
4. Добавьте специфичные для проекта команды автотеста.
5. Все ДО должны поддерживать `--autotest` — это обязательное требование.